

Pontificia Universidad Javeriana

Taller de Diseño en Software
Proyecto Leadify

Integrantes: Juan Diego Ortiz Guerrero, Carlos Andrés Figueroa Vega, Santiago Díaz Sabogal y Sara Cuervo Avendaño

20/10/2026

1. Introducción

Leadify convierte a los visitantes anónimos del sitio de una constructora en citas agendadas con un asesor comercial. La persona responde un cuestionario visual de 7 preguntas mientras ve armarse en pantalla el plano de su apartamento. Un motor de recomendación (Nearest Neighbors con una capa económica) le muestra los proyectos más compatibles con su perfil y su capacidad de pago y, al final, la agente de voz Manuela (Dapta) la llama para resolver dudas y agendar la cita.

El sistema está en producción: el formulario corre en Vercel, la API y el motor en Render, los leads se guardan en Supabase (PostgreSQL) y las llamadas las hace Dapta. El diseño de este taller parte de las 3 historias de usuario más importantes del producto:

Historia	Enunciado	Por qué es crítica
HU-A (principal)	Como comprador, quiero responder un cuestionario visual y recibir los proyectos que mejor encajan con mi perfil y mi capacidad de pago, con su porcentaje de compatibilidad.	Es la propuesta de valor: sin recomendación no hay lead que calificar ni llamada que hacer.
HU-B	Como constructora, quiero que cada visitante quede registrado y calificado ("Listo para asesor" o "En maduración"), y que al volver vea sus resultados sin repetir el cuestionario.	Convierte al visitante anónimo en un lead identificado y priorizado.
HU-C	Como comprador, quiero que Manuela me llame sobre el proyecto que elegí para resolver dudas y agendar una cita con un asesor.	Cierra el embudo: lleva al lead hasta la cita.

Con esas tres historias se construyó un único conjunto de cuatro diagramas: análisis (ECB), clases, componentes y despliegue. Cada uno parte del anterior y todos se hicieron a partir del código real del repositorio. En el modelo de vistas 4+1, el análisis ECB y el diagrama de clases forman la vista lógica, el diagrama de componentes la vista de implementación y el de despliegue la vista física.

Los diagramas en tamaño completo, con su código fuente en PlantUML, están en la carpeta `Docs/Diagramas` del repositorio: github.com/santiagodiazsabogal/Leadify.

2. issues y sus commits

Cada diagrama tiene su propio sub-issue, vinculado a la historia principal HU-A en el tablero del repositorio, y se subió a la carpeta `Docs/Diagramas` en formato Markdown con un commit que referencia y cierra ese issue.

Sub-issue	Prioridad	Asignado a	Sprint	Commit asociado
Diagrama de análisis (ECB)	Alta	Sara Cuervo	Sprint 4	Agregar diagrama de análisis ECB de las 3 HU críticas
Diagrama de clases	Alta	Carlos Figueroa	Sprint 4	Agregar diagrama de clases con principios SOLID
Diagrama de componentes	Media	Santiago Díaz	Sprint 4	Agregar diagrama de componentes con interfaces provistas y requeridas
Diagrama de despliegue	Media	Juan Diego Ortiz	Sprint 4	Agregar diagrama de despliegue con nodos, artefactos y puertos

3. Diagrama de Análisis (ECB)

El diagrama de análisis reparte cada historia crítica en límites (`«boundary»`), controles (`«control»`) y entidades (`«entity»`), sin hablar todavía de tecnología. Hay un panel por historia, con el comprador arriba y las entidades abajo.

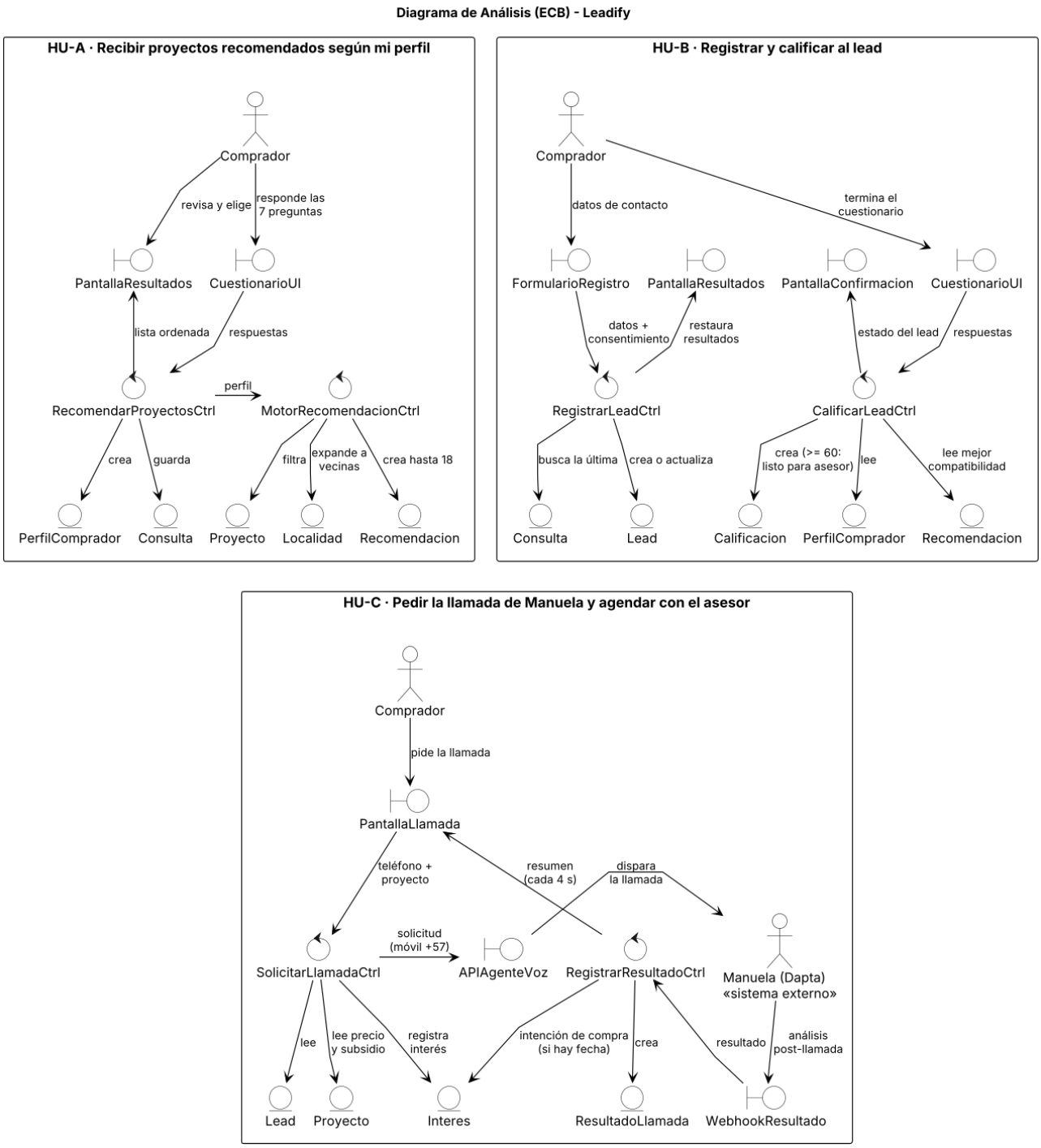


Figura 1. Diagrama de análisis ECB de las 3 historias críticas (01-diagrama-de-analisis-ecb.md).

Historia	**Límites (boundary)**	**Controles (control)**	**Entidades (entity)**
HU-A · Recomendación	CuestionarioUI, PantallaResultados	RecomendarProyectosCtrl, MotorRecomendacionCtrl	PerfilComprador, Consulta, Proyecto, Localidad, Recomendacion
HU-B · Registro y calificación	FormularioRegistro, CuestionarioUI, PantallaResultados, PantallaConfirmacion	RegistrarLeadCtrl, CalificarLeadCtrl	Lead, Consulta, PerfilComprador, Recomendacion, Calificacion
HU-C · Llamada de Manuela	PantallaLlamada, APIAgenteVoz, WebhookResultado	SolicitarLlamadaCtrl, RegistrarResultadoCtrl	Lead, Proyecto, Interes, ResultadoLlamada

El diagrama respeta las reglas de robustez: los actores solo hablan con límites, los límites solo con controles, y solo los controles leen o modifican entidades. Dapta, que es un sistema externo, aparece como actor y se alcanza por dos límites propios: `APIAgenteVoz` para pedir la llamada y `WebhookResultado` para recibir su análisis.

También quedan a la vista tres reglas de negocio: el motor devuelve hasta 18 proyectos; el lead es "Listo para asesor" con 60 puntos o más, sobre una base de 40; y la intención de compra solo se marca si la llamada termina con fecha de seguimiento.

4. Diagrama de Clases

El diagrama de clases es el modelo de diseño que sale del análisis: los controles pasan a ser controladores y servicios, las entidades a clases del dominio y los límites hacia sistemas externos a interfaces. Está organizado en seis paquetes: Controladores, Dominio, Recomendación, Calificación, Llamadas y Persistencia. Cada clase tiene nombre en singular, atributos con tipo y visibilidad (`-cedula: String`, `#nombre: String`) y métodos con parámetros y tipo de retorno (`+calificar(consulta: Consulta): CalificacionLead`).

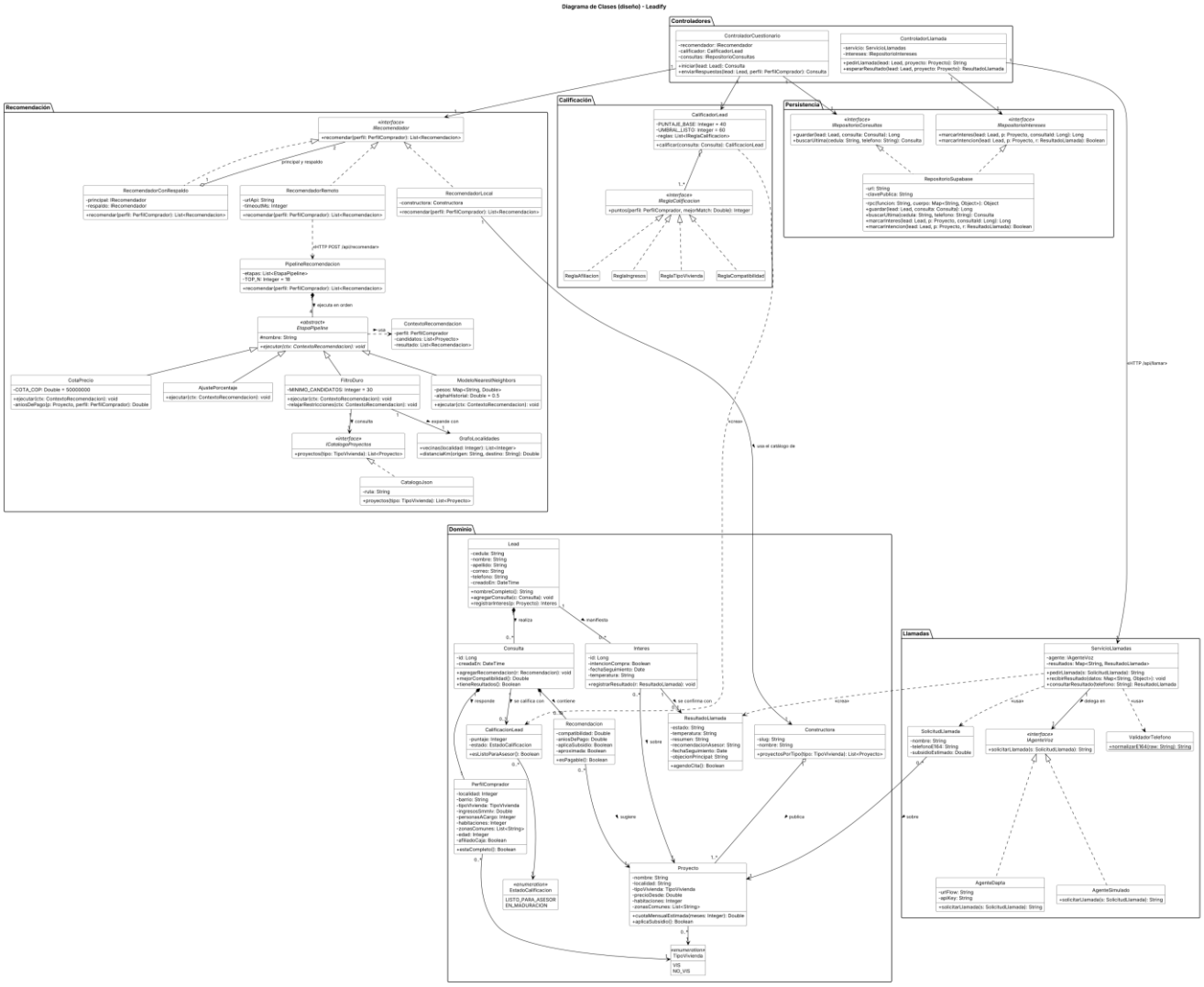


Figura 2. Diagrama de clases completo (02-diagrama-de-clases.md). Las figuras 2a a 2d amplían cada paquete.

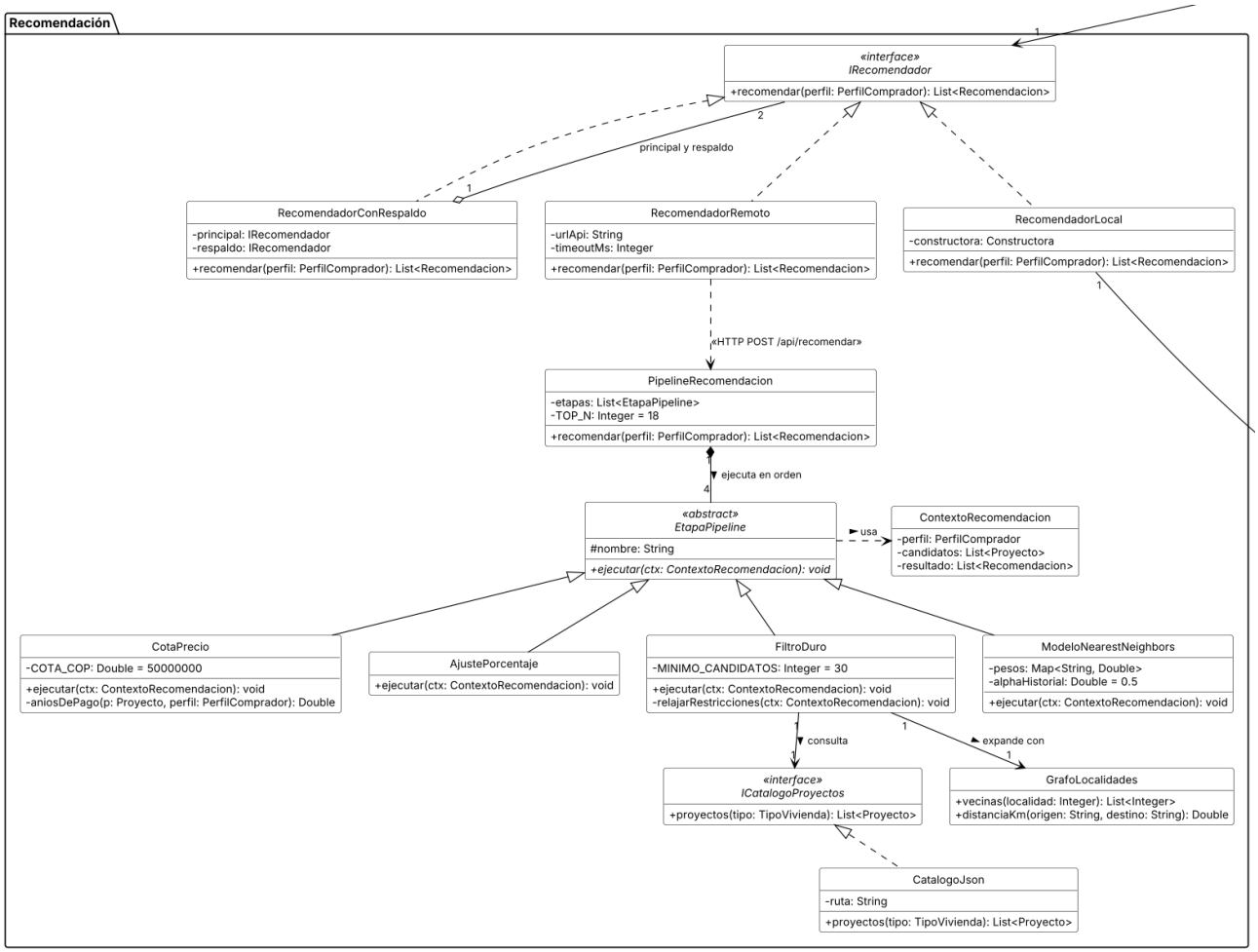


Figura 2a. Paquete Recomendación: interfaz IRecomendador con sus tres implementaciones y el pipeline del motor con sus cuatro etapas.

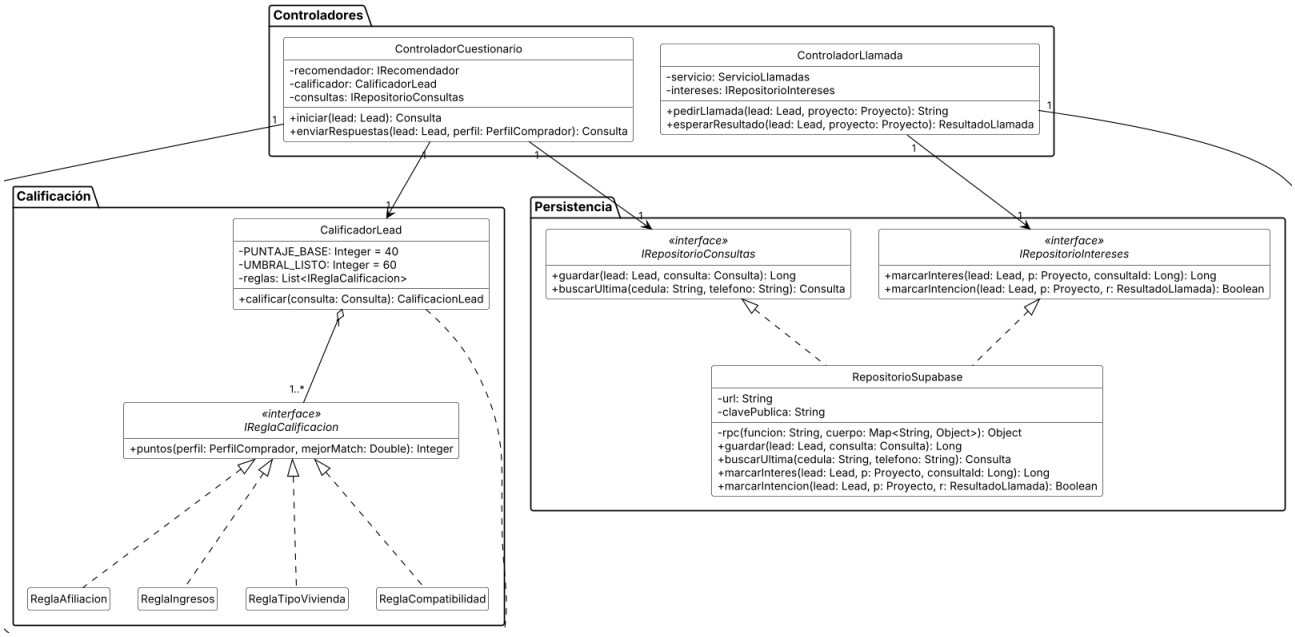


Figura 2b. Paquetes Controladores, Calificación y Persistencia.

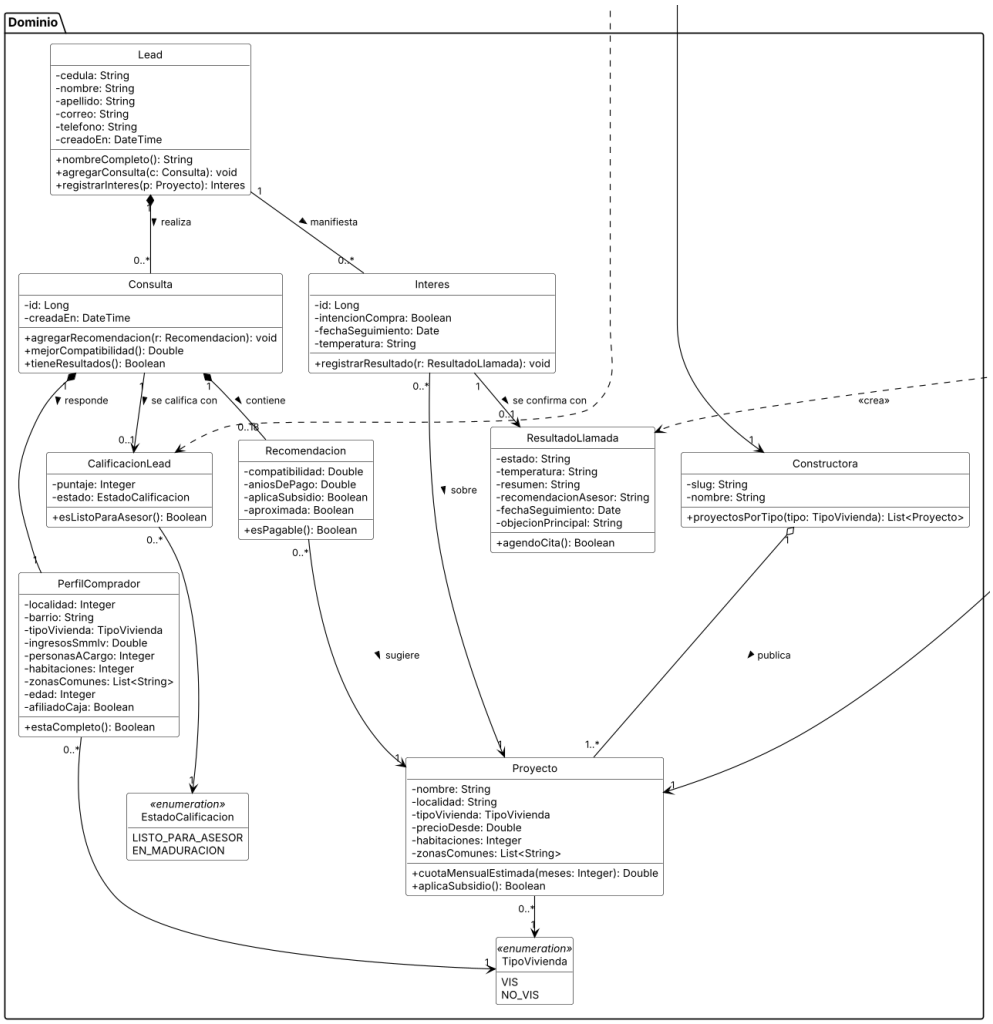


Figura 2c. Paquete Dominio: composición, agregación, asociaciones y multiplicidades.

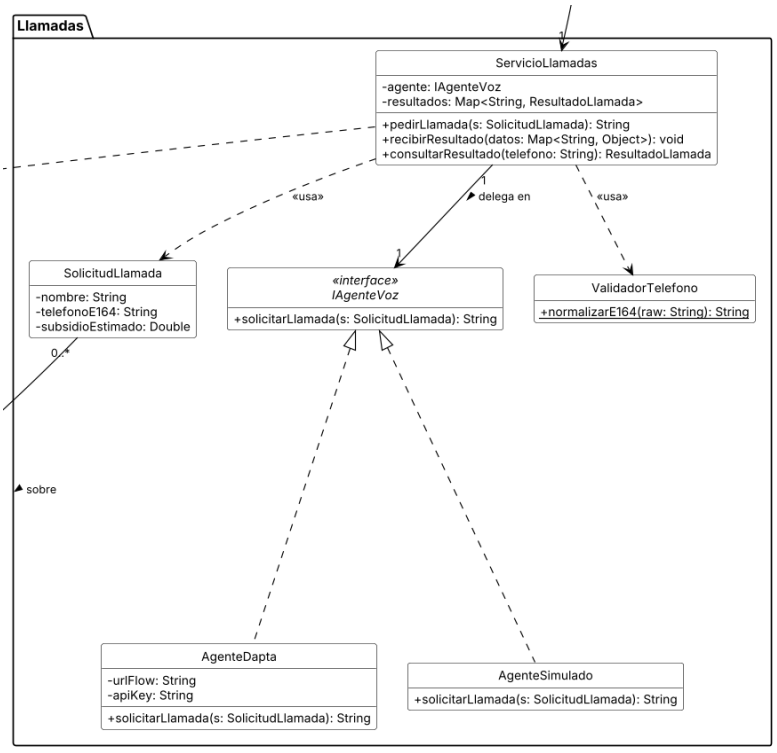


Figura 2d. Paquete Llamadas.

4.1. Relaciones

Relación	Ejemplo en el diagrama	Multiplicidad
Herencia	`FiltroDuro`, `ModeloNearestNeighbors`, `CotaPrecio` y `AjustePorcentaje` heredan de la clase abstracta `EtapaPipeline`.	—
Realización de interfaz	`RecomendadorRemoto`, `RecomendadorLocal` y `RecomendadorConRespaldo` implementan `IRecomendador`; `AgenteDapta` y `AgenteSimulado` implementan `IAgenteVoz`.	—
Composición	`Lead` ◆— `Consulta`: si se borra el lead se borran sus consultas. `Consulta` ◆— `Recomendacion` y `Consulta` ◆— `PerfilComprador`.	1 a 0..*; 1 a 0..18; 1 a 1
Agregación	`Constructora` ◇— `Proyecto`: el proyecto existe aunque cambie la marca. `CalificadorLead` ◇— `IReglaCalificacion`.	1 a 1..*
Asociación	`Lead` — `Interes` → `Proyecto`; `Consulta` → `CalificacionLead`; cada controlador con sus servicios.	1 a 0..*; 0..* a 1; 1 a 0..1
Dependencia	`CalificadorLead` crea `CalificacionLead`; `ServicioLlamadas` usa `ValidadorTelefono`.	—

4.2. Principios SOLID

Principio	Cómo se aplica
S · Responsabilidad única	Cada clase tiene una sola razón para cambiar: si cambia la fórmula de calificación solo cambia `CalificadorLead`; si cambia el formato del teléfono, solo `ValidadorTelefono`; si cambia la base de datos, solo `RepositorioSupabase`.
O · Abierto/cerrado	Para agregar una etapa al motor, una regla de puntaje o un proveedor de llamadas distinto de Dapta se crea una clase nueva que implemente `EtapaPipeline`, `IReglaCalificacion` o `IAgenteVoz`, sin modificar las existentes.
L · Sustitución de Liskov	El motor local reemplaza al remoto sin que el controlador lo note (devuelve la misma lista, marcada como aproximada), y `AgenteSimulado` reemplaza a `AgenteDapta` en pruebas.
I · Segregación de interfaces	Hay dos interfaces de persistencia pequeñas en vez de una grande: `ControladorCuestionario` solo usa `IRepositorioConsultas` y `ControladorLlamada` solo `IRepositorioIntereses`.
D · Inversión de dependencias	Los controladores y servicios dependen de abstracciones (`IRecomendador`, `IRepositorioConsultas`, `IAgenteVoz`), no de Supabase ni de Dapta.

En el diseño también se ven los patrones vistos en clase: GRASP Controlador (`ControladorCuestionario`), Experto (`Consulta.mejorCompatibilidad()`) y Creador (`Lead.registrarInteres()`), y los patrones GoF Strategy (`IRecomendador`), Facade (`PipelineRecomendacion`) y Template Method (`EtapaPipeline.ejecutar()`). Además respeta MVC: los controladores reciben la acción del usuario y deciden a quién delegar, el modelo son las clases del dominio y los servicios, y la vista solo muestra lo que el controlador le entrega.

5. Diagrama de Componentes

El sistema se divide en dos subsistemas, Leadify Web (corre en el navegador) y Leadify API (Python con FastAPI), y cuatro servicios externos. Los componentes se comunican solo a través de interfaces: el círculo es una interfaz provista y el semicírculo una interfaz requerida. Los servicios externos se dibujan como base de datos o nube, fuera de los subsistemas y con el estereotipo «`servicio externo`» o «`API externa`».

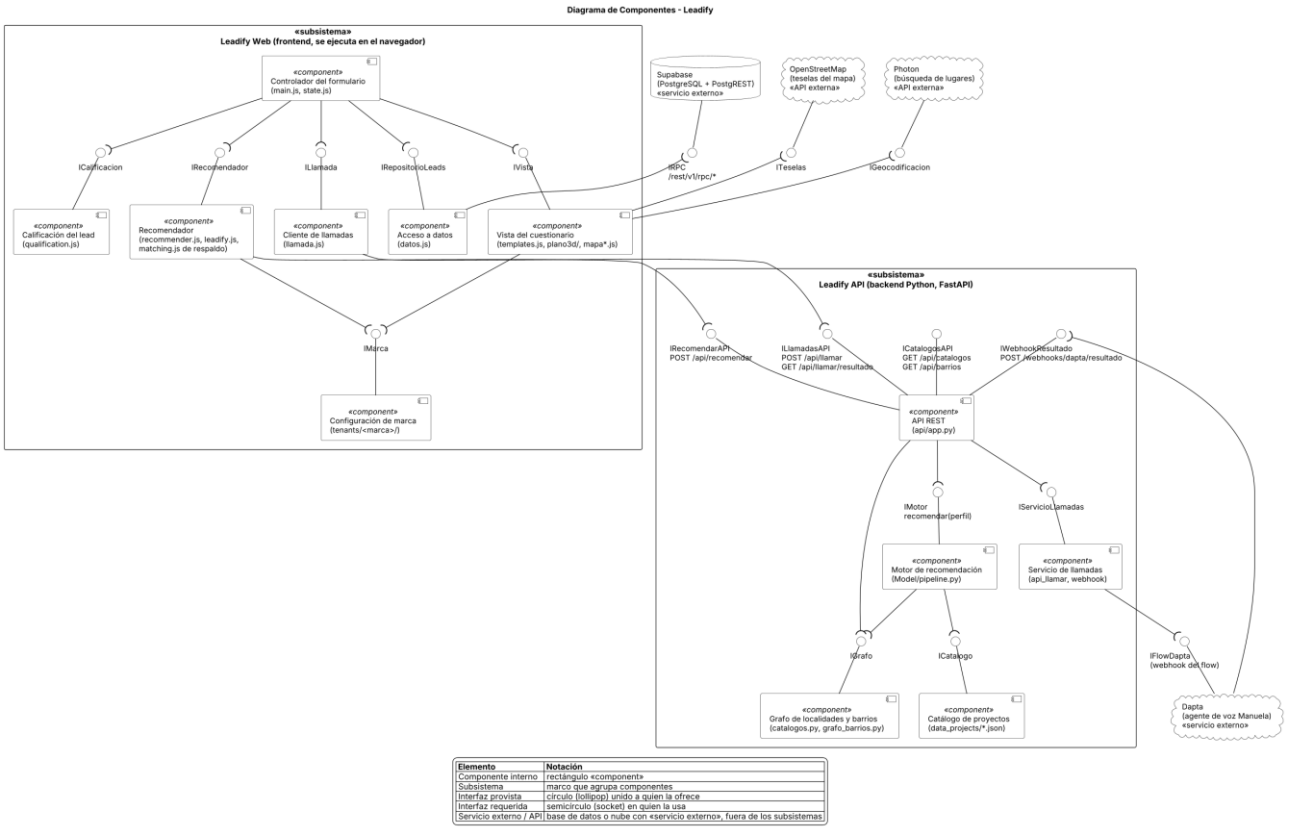


Figura 3. Diagrama de componentes completo (03-diagrama-de-componentes.md).

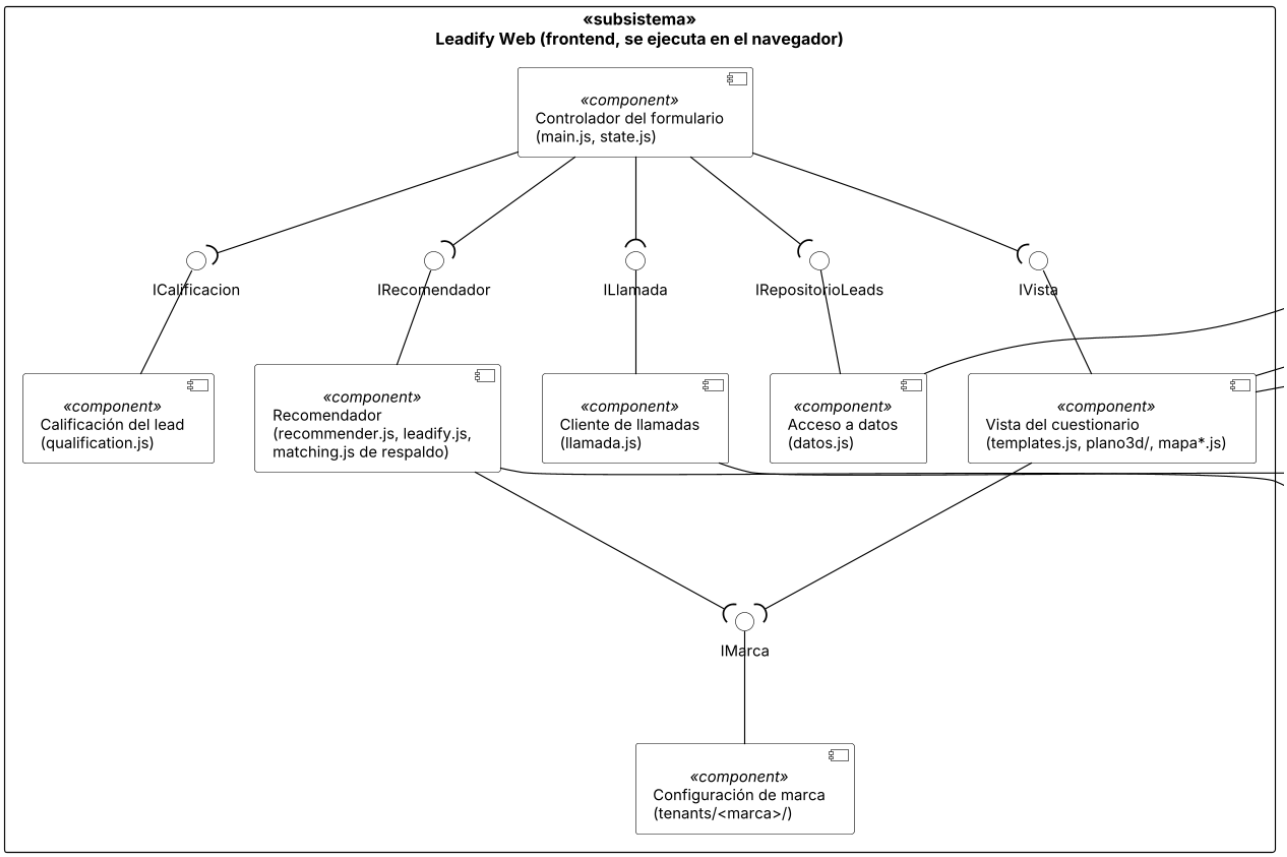


Figura 3a. Subsistema Leadify Web.

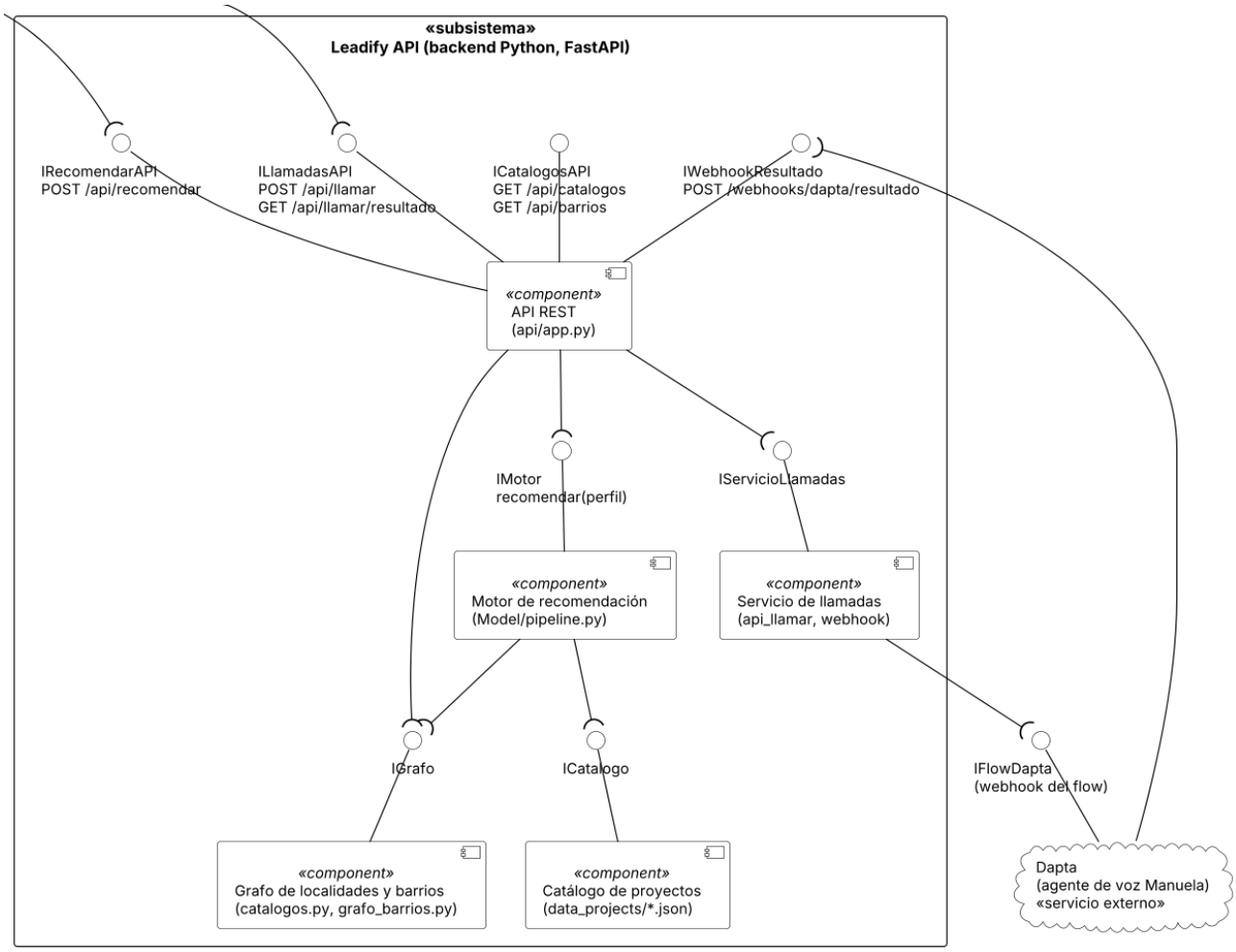


Figura 3b. Subsistema Leadify API y su conexión con Dapta.

Componente	**Provee**	**Requiere**
Controlador del formulario (`main.js`, `state.js`)	—	IVista, IRecomendador, ICalificacion, IRepositorioLeads, ILlamada
Vista del cuestionario (`templates.js`, `plano3d/`, `mapa*.js`)	IVista	IMarca, IGeocodificacion, ITeselas
Recomendador (`recommender.js`, `leadify.js`, `matching.js`)	IRecomendador	IRecomendarAPI, IMarca
Calificación del lead (`qualification.js`)	ICalificacion	—
Acceso a datos (`datos.js`)	IRepositorioLeads	IRPC de Supabase
Cliente de llamadas (`llamada.js`)	ILlamada	ILlamadasAPI
Configuración de marca (`tenants/<marca>/`)	IMarca	—
API REST (`api/app.py`)	IRecomendarAPI, ICatalogosAPI, ILlamadasAPI, IWebhookResultado	IMotor, IServicioLlamadas, IGrafo
Servicio de llamadas	IServicioLlamadas	IFlowDapta
Motor de recomendación (`Model/pipeline.py`)	IMotor	ICatalogo, IGrafo
Catálogo de proyectos y grafo de localidades	ICatalogo, IGrafo	—

El diagrama refleja tres decisiones de arquitectura. Primero, el estilo en N capas en niveles separados (presentación desacoplada): presentación en el navegador, lógica en la API y datos en Supabase, comunicados por REST/JSON. Segundo, el formulario sigue MVC: la Vista solo muestra, el Controlador decide y el resto de componentes son el modelo. Tercero, el navegador nunca habla con Dapta: la llamada pasa por la API, que es la única que conoce la clave secreta.

6. Diagrama de Despliegue

Leadify corre en cinco nodos de nube más el dispositivo del comprador, y todo el tráfico público viaja por HTTPS en el puerto 443. Cada nodo es un cubo 3D con sus artefactos («`artifact`») adentro.

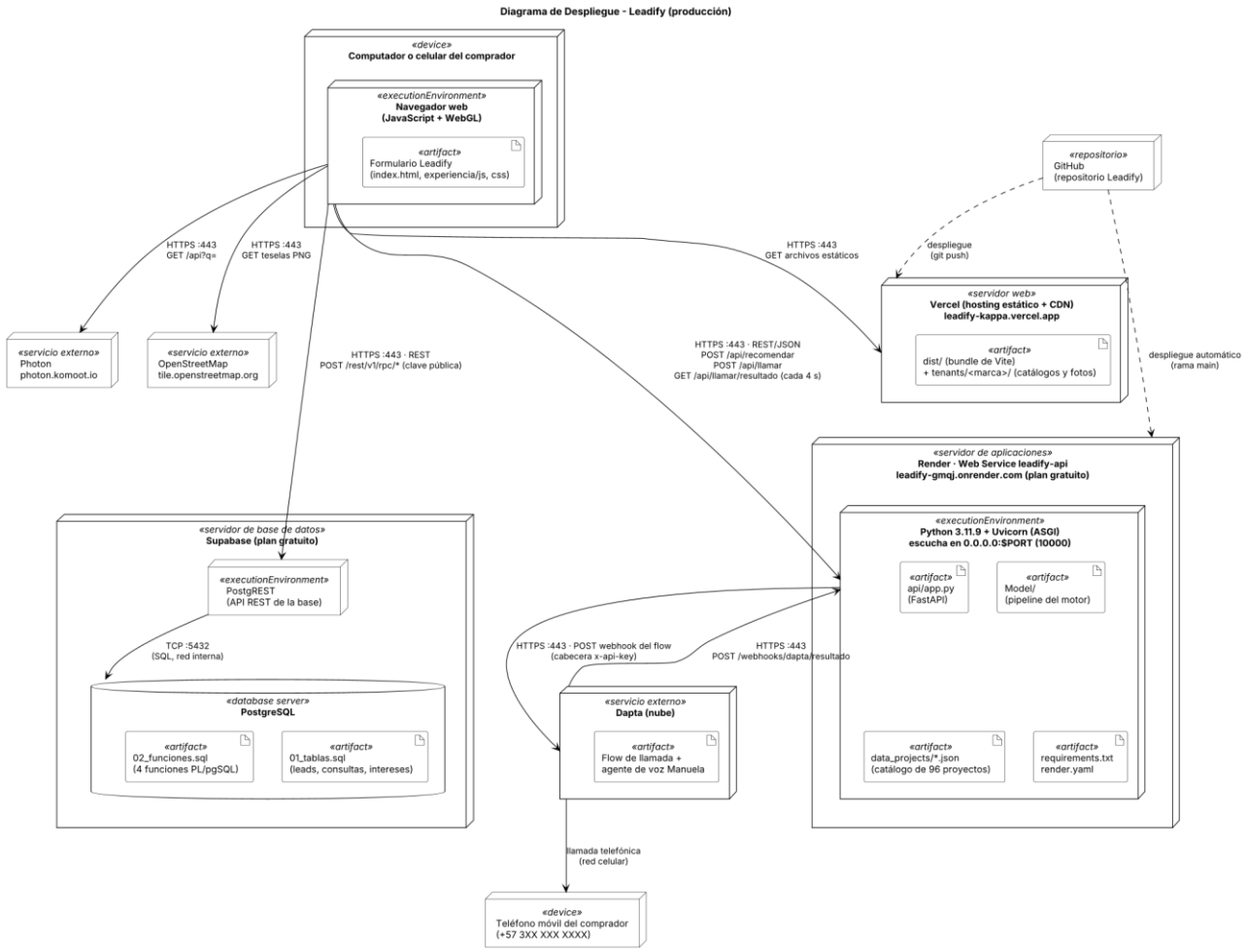


Figura 4. Diagrama de despliegue en producción (04-diagrama-de-despliegue.md).

6.1. Comunicaciones

Origen	Destino	Protocolo y puerto	Para qué
Navegador	Vercel	HTTPS :443	Descargar el formulario (bundle de Vite y catálogos de cada marca)
Navegador	Render (Uvicorn)	HTTPS :443, REST/JSON	`POST /api/recomendar`, `POST /api/llamar` y `GET /api/llamar/resultado` cada 4 s
Navegador	Supabase (PostgREST)	HTTPS :443, REST	`POST /rest/v1/rpc/<función>` con la clave pública
PostgREST	PostgreSQL	TCP :5432	Consultas SQL en la red interna de Supabase
Render	Dapta	HTTPS :443	Disparar el flow de llamada con la cabecera `x-api-key`
Dapta	Render	HTTPS :443	`POST /webhooks/dapta/resultado` con el análisis de la llamada

Origen	Destino	Protocolo y puerto	Para qué
Dapta	Teléfono del comprador	Red celular	La llamada de Manuela
Navegador	Photon / OpenStreetMap	HTTPS :443	Búsqueda de lugares y teselas del mapa

6.2. Infraestructura y hardware necesario

Nodo	Hardware y software mínimo
Dispositivo del comprador	Navegador actualizado con JavaScript y WebGL (el plano 3D usa three.js); no requiere instalación.
Vercel	Hosting estático con CDN; Node.js 20.19 o superior para compilar el bundle.
Render (plan gratuito)	512 MB de RAM y Python 3.11.9 con Uvicorn en `0.0.0.0:$PORT` (10000 por defecto); se duerme tras 15 minutos sin tráfico y la primera petición puede tardar cerca de un minuto.
Supabase (plan gratuito)	500 MB de base de datos, CPU compartida y 500 MB de RAM; se pausa tras una semana sin actividad; RLS activo.
Dapta	Cuenta con créditos de llamada y un flow configurado con su webhook.
Teléfono del comprador	Móvil colombiano (+57 y 10 dígitos que empiezan por 3).

GitHub es el origen del despliegue: Render publica la API desde la rama `main`, como lo define `render.yaml`, y Vercel publica el formulario. Los límites de los planes gratuitos se tomaron de la documentación de Render (render.com/docs/free, render.com/pricing y render.com/docs/web-services) y de la página de precios de Supabase (supabase.com/pricing).

7. Conclusiones

Modelar el sistema después de construirlo nos obligó a hacer explícitas decisiones que en el código estaban implícitas, y dejó a la vista puntos concretos para mejorar:

- **El análisis ECB ordenó las responsabilidades:** al separar límites, controles y entidades se vio que la lógica de negocio (recomendar, calificar, validar el teléfono) no debe vivir en las pantallas. Eso coincide con lo que ya hace el código: `state.js` no hace entrada ni salida y cada cálculo está en su propio módulo.
- **El diagrama de clases tradujo funciones a un diseño orientado a objetos:** el código está escrito en funciones y módulos, pero ya seguía la idea de los principios SOLID. El recomendador elige entre el motor remoto y el local sin que el resto del formulario lo note (Strategy, Liskov), y el motor es una cadena de etapas que se pueden cambiar por separado (abierto/cerrado).
- **El diagrama de componentes mostró los contratos reales:** al dibujar cada interfaz provista y requerida encontramos que la API ofrece `GET /api/catalogos` y `GET /api/barrios`, pero el formulario hoy no los usa porque trae sus propias listas. Son dos copias de los mismos datos que pueden desincronizarse.
- **El diagrama de despliegue hizo visibles los límites de la infraestructura:** el plan gratuito de Render se duerme tras 15 minutos y los resultados de las llamadas se guardan en memoria, así que se pierden si el servicio se reinicia. También se vio que la calificación del lead se calcula en el navegador pero no se guarda en la base de datos, por lo que el asesor no la puede consultar después.

Como siguiente paso, estos hallazgos se pueden convertir en historias de usuario: guardar la calificación en la tabla de leads, persistir los resultados de las llamadas en Supabase y usar los catálogos de la API como única fuente.